

Запускать так: `./run.sh`.

Попытаемся найти перестановку вершин, на которой при жадном алгоритме выходит мало цветов. Далее, подобно эвристике с лекции будем переставлять группы цветов и перезапускать жадный алгоритм. После того как сойдёмся начнём локальный поиск по покраске. Теперь обо всём по порядку:

Рассмотрим перестановку вершин $1, 2, \dots, n$. Для неё запустим жадный алгоритм: будем красить вершины в порядке перестановки так, чтобы не возникало противоречий с уже покрашенными. После того как граф будет покрашен, рассмотрим размер каждой получившейся группы вершин одного цвета и переставим вершины в таком порядке, что сначала будут идти вершины самого частого цвета, потом цвета чуть реже, и так далее, в конце будет самый редкий цвет. На получившейся перестановке снова запустим тот же жадный алгоритм. Так будем повторять до сходимости.

В дословно такой реализации получились следующие значения: 9, 23, 23, 100, 20, 122. Что дало 3 балла следующим образом: 0, 0, 0, 0, 0, 3.

Таким образом мы хотели получить раскраску, у которой популярность цветов будет в отсортированном порядке, тем самым избежать часть симметрий. Проблема такого метода заключается в том, что работать до сходимости на самом деле оказывается не очень долго, а именно было потрачено 1, 1, 2, 1, 3, 5 итераций на тест соответственно.

Также было попробовано запускаться из множества случайных начальных перестановок, но даже 10000 случайно выбранных перестановок не дали улучшения, все значения числа цветов на тестах остались прежними.

Также была попробована и другая сортировка, а именно сортировать по убыванию степеней вершин, она дала значительно лучший результат, получились метрики: 7, 20, 21, 97, 18, 121. То есть 15 баллов следующим образом: 3, 3, 3, 0, 3, 3.

После этого был написан ещё один конструктив.

Введём следующее отношение порядка на ответах на покраску: выпишем массив того, сколько вершин какого цвета у нас есть и отсортируем его от большего к меньшему. Будем считать раскраски в которых в последнем классе меньше вершин лучше чем другие раскраски в такое же количество цветов. Мотивация этого: требуется найти другой цвет для меньшего числа вершин. Уже имеющаяся эвристика проталкивает вершины в более ранние цветовые классы в порядке сортировки, так что будем считать, что она минимизирует самый последний.

Научимся перекрашивать вершину из последнего класса: возьмём самую последнюю вершину v в расстановке и посмотрим на всех её соседей. Нам нужно найти новый цвет для v , а если v цвета k , то у неё есть соседи всех меньших цветов. Тогда для каждого из цветов $1, 2, \dots, k - 1$ возьмём все вершины из окружения v этого цвета и проверим, может быть их можно перекрасить в какой-то другой цвет. Для этого достаточно перекрасить каждую по отдельности, так как сами вершины одного цвета в корректной покраске, а значит образуют независимое множество, то есть их новые цвета не могут давать противоречие в раскраске между друг-другом, только с остальными вершинами. Если так вышло, что всех соседей v одного какого-то цвета, скажем i , можно перекрасить, то перекрасим v в цвет i , а всех соседей v цвета i в тот, который был найден как возможный, получим раскраску лучше, снова передадим её нашей эвристике, покрасим набор и повторим процедуру. Если процедура остановилась, то попробуем и для других вершин последнего цвета. Если и для них ничего нельзя изменить, то остановимся.

Этот подход улучшил метрики, вышли следующие результаты: 7, 20, 19, 94, 17, 113. То есть 18 баллов, следующим образом: 3, 3, 3, 3, 3, 3. Заметим, что на всех тестах этот процесс со-

шёлся, то есть проделывать такие улучшения больше было нельзя.

Далее был добавлен *Beam Search*. Запустив все наши эвристики найдём какую-нибудь стартовую точку, из неё начнём идти с помощью *Beam Search*. Выберем метрику, как и говорилось ранее, следующим образом: посмотрим на массив количеств всех цветов. Сперва сравним его по размеру, если они не равны, то меньше цветов — лучше. Если количества цветов равны, то лучше тот, у кого меньше вершин в наименее популярном цвете, при равенстве — в цвете на 1 популярнее и так далее. Изменения будем делать следующим образом: возьмём вершину последнего цвета и попытаемся поменять её с кем-то из её соседей (изменение в распределении цветов будет только если в перестановки свапнуть смежные вершины, а в первую очередь хочется избавиться от последнего цвета). Если изменение улучшает разбиение то сразу же примем его, иначе закинем все возможные разбиения после 1 такого шага в кучу, где если мы поменяли (u, v) , то запомним, что надо сделать *swar* вершины u с кем-то, или вершины v с кем-то, в конце выделим из кучи несколько самых лучших найденных разбиений, и продолжим поиск для них, если после пути некоторой длины (выбрана 5) не удалось ничего улучшить, то будем считать что мы в локальном минимуме и остановимся.

Этот подход улучшил метрики на средних тестах (на пятом и шестом при выбранных константах он не сумел сделать один шаг, уложившись во время). Метрики составили: 7, 18, 18, 85, 17, 113. То есть всё те же 18 баллов 3, 3, 3, 3, 3, 3.